Lab 3: Advanced Routing and Request Handling

In the previous lab, your application relied on ASP.NET Core's default convention-based routing (the `{controller}/{action}/{id?}` pattern defined in `Program.cs`).

To build robust applications, you need explicit control over your URLs, how parameters are constrained, and how different types of requests are handled.

Here is how to upgrade your `BooksController` using Attribute Routing, Route Constraints, and custom Request Handlers.

1. Implement Attribute Routing at the Controller Level

Overrides default conventions for exact URL control. Open `Controllers/BooksController.cs`. Add the `[Route]` attribute to the class level to establish a base URL for all actions inside this controller.

C# – Controllers/BooksController.cs

```
using Microsoft.AspNetCore.Mvc;
using MyMvcApp.Models;
using System.Collections.Generic;
using System.Linq;

namespace MyMvcApp.Controllers
{
    // 1. Establish the base route for this controller
    [Route("Library/Books")]
    public class BooksController : Controller
    {
        private static List<Book> _books = new List<Book>();
        private static int _nextId = 1;

        // 2. Map this action to the base route exactly (/Library/Books)
        [HttpGet("")]
        public IActionResult Index() => View(_books);

        // 3. Map to /Library/Books/New
        [HttpGet("New")]
        public IActionResult Create() => View();

        [HttpPost("New")]
        public IActionResult Create(Book book)
        {
            book.Id = _nextId++;
            _books.Add(book);
            return RedirectToAction("Index");
        }
        // ... (keep the rest of the controller intact for now)
    }
}
```

Note: Because we changed the base URL to `Library/Books`, you must update your `_Layout.cshtml` navigation link from `asp-controller="Books"` to a direct href: `<a class="nav-link`

text-dark" href="/Library/Books">Books Directory.

2. Add Route Constraints to Parameterized Views

Prevents invalid data types from hitting your logic. Currently, navigating to `/Books/Edit/abc` attempts to pass a string into an integer ID, causing an error. Route constraints ensure a route only matches if the parameter is the correct type.

Update your `Edit` and `Delete` actions in `BooksController.cs`:

C# – Edit and Delete Actions

```
// Map to /Library/Books/5/Edit. The ":int" forces the ID to be a number.
[HttpGet("{id:int}/Edit")]
public IActionResult Edit(int id)
{
    var book = _books.FirstOrDefault(b => b.Id == id);
    if (book == null) return NotFound(); // Returns a 404 status code
    return View(book);
}

[HttpPost("{id:int}/Edit")]
public IActionResult Edit(int id, Book updatedBook)
{
    // Request handling: Validate that the route ID matches the posted model
ID
    if (id != updatedBook.Id) return BadRequest(); // Returns a 400 status
code

    var book = _books.FirstOrDefault(b => b.Id == updatedBook.Id);
    if (book != null)
    {
        book.Title = updatedBook.Title;
        book.Author = updatedBook.Author;
    }
    return RedirectToAction("Index");
}

[HttpGet("{id:int}/Delete")]
public IActionResult Delete(int id)
{
    var book = _books.FirstOrDefault(b => b.Id == id);
    if (book != null) _books.Remove(book);
    return RedirectToAction("Index");
}
```

3. Handle Query Strings for Searching

Process optional URL parameters. Not all data should be part of the route path. For things like search filters, we use Query Strings (e.g., `?author=Tolkien`).

Add a new Search action to your controller that handles requests from the `[FromQuery]` binding:

C# – Query String Search Action

```
// Map to /Library/Books/Search?author=Name
[HttpGet("Search")]
public IActionResult Search([FromQuery] string author)
{
    if (string.IsNullOrWhiteSpace(author))
    {
        // If no search term is provided, redirect back to the full list
        return RedirectToAction("Index");
    }

    var filteredBooks = _books
        .Where(b => b.Author.Contains(author, System.StringComparison.
OrdinalIgnoreCase))
        .ToList();

    // We can reuse the existing Index view to display the filtered list
    return View("Index", filteredBooks);
}
```

4. Diversify the Action Result Types

Respond with JSON instead of HTML views. MVC controllers aren't restricted to returning web pages. You can handle requests from mobile apps or JavaScript frontends by returning raw data.

Add an API endpoint to your controller that bypasses the View engine entirely:

C# – API Action returning JSON

```
// Map to /Library/Books/api/all
[HttpGet("api/all")]
public IActionResult GetBooksApi()
{
    // Returns a 200 OK with the list of books automatically serialized to
JSON
    return Ok(_books);
}
```

Detailed Instructions for Lab 3: Implementing Advanced Routing and Request Handling in MVC

To demonstrate how to gain explicit control over your application's URLs and data flow, we will upgrade the Book CRUD app from Lab 2. We will replace the default routing conventions with Attribute Routing, implement Route Constraints to prevent errors, handle Query Strings for a search feature, and create an endpoint that returns raw JSON.

Here is how to manually wire up these routing and request-handling features.

Implement Attribute Routing & Constraints

Defines explicit URLs and strict parameter rules.

In VS Code, open your existing `Controllers/BooksController.cs` file. We are going to replace its contents.

By adding `[Route("Library/Books")]` to the top of the class, we change the base URL. By adding `{id:int}` to our Edit and Delete actions, we ensure the application returns a 404 (Not Found) if a user tries to pass text instead of a valid number in the URL (e.g., `/Library/Books/abc/Edit`).

Replace the code in `BooksController.cs` with this updated version:

C# – Controllers/BooksController.cs (Complete)

```
using Microsoft.AspNetCore.Mvc;
using MyMvcApp.Models;
using System.Collections.Generic;
using System.Linq;

namespace MyMvcApp.Controllers
{
    // Establishes a custom base URL for every action in this controller
    [Route("Library/Books")]
    public class BooksController : Controller
    {
        private static List<Book> _books = new List<Book>();
        private static int _nextId = 1;

        // READ: Matches exactly GET /Library/Books
        [HttpGet("")]
        public IActionResult Index() => View(_books);

        // CREATE: Matches GET /Library/Books/New
        [HttpGet("New")]
        public IActionResult Create() => View();

        // CREATE: Matches POST /Library/Books/New
        [HttpPost("New")]
        public IActionResult Create(Book book)
        {
            book.Id = _nextId++;
            _books.Add(book);
            return RedirectToAction("Index");
        }

        // UPDATE: Matches GET /Library/Books/5/Edit
        // The ":int" constraint ensures this only fires if the ID is a number
        [HttpGet("{id:int}/Edit")]
        public IActionResult Edit(int id)
        {
            var book = _books.FirstOrDefault(b => b.Id == id);
            if (book == null) return NotFound();
            return View(book);
        }

        // UPDATE: Matches POST /Library/Books/5/Edit
        [HttpPost("{id:int}/Edit")]
        public IActionResult Edit(int id, Book updatedBook)
        {
            // Request handling: Ensure the URL ID matches the submitted form ID
            if (id != updatedBook.Id) return BadRequest();

            var book = _books.FirstOrDefault(b => b.Id == updatedBook.Id);
            if (book != null)
            {
                book.Title = updatedBook.Title;
                book.Author = updatedBook.Author;
            }
            return RedirectToAction("Index");
        }
```

C# – Controllers/BooksController.cs

```
        // DELETE: Matches GET /Library/Books/5/Delete
        [HttpGet("{id:int}/Delete")]
        public IActionResult Delete(int id)
        {
            var book = _books.FirstOrDefault(b => b.Id == id);
            if (book != null) _books.Remove(book);
            return RedirectToAction("Index");
        }

        // SEARCH: Matches GET /Library/Books/Search?author=Name
        // Uses [FromQuery] to handle optional URL parameters
        [HttpGet("Search")]
        public IActionResult Search([FromQuery] string author)
        {
            if (string.IsNullOrWhiteSpace(author))
            {
                return RedirectToAction("Index");
            }

            var filteredBooks = _books
                .Where(b => b.Author.Contains(author, System.StringComparison.
    OrdinalIgnoreCase))
                .ToList();

            // Reuses the Index view to display the filtered results
            return View("Index", filteredBooks);
        }

        // API: Matches GET /Library/Books/api/all
        // Returns raw JSON data instead of an HTML view
        [HttpGet("api/all")]
        public IActionResult GetBooksApi()
        {
            return Ok(_books);
        }
    }
}
```

Update the Index View

The 'Read' and 'Search' interface.

To use our new Search method, we need to add an HTML form that submits a GET request. Because we used standard Tag Helpers (`asp-action`), ASP.NET Core will automatically read our controller's `[Route]` attributes and generate the correct URLs behind the scenes.

Open `/Views/Books/Index.cshtml` and update it to match this code:

HTML / Razor – /Views/Books/Index.cshtml

```
@model IEnumerable<MyMvcApp.Models.Book>

<h2>Book Directory</h2>

<!-- Search Form targeting the new Search action -->
<form asp-action="Search" method="get" class="mb-3 d-flex gap-2">
    <input type="text" name="author" class="form-control w-25" placeholder="
    Search by author..." />
    <button type="submit" class="btn btn-secondary">Search</button>
    <a asp-action="Index" class="btn btn-outline-secondary">Clear</a>
</form>

<a asp-action="Create" class="btn btn-primary mb-3">Add New Book</a>

<table class="table table-striped">
    <tr>
        <th>Title</th>
        <th>Author</th>
        <th>Actions</th>
    </tr>
    @foreach (var book in Model)
    {
        <tr>
            <td>@book.Title</td>
            <td>@book.Author</td>
            <td>
                <a asp-action="Edit" asp-route-id="@book.Id">Edit</a> |
                <a asp-action="Delete" asp-route-id="@book.Id" class="text-danger
    ">Delete</a>
            </td>
        </tr>
    }
</table>
```

Update the Navigation Bar

Adjusting main menu links for Attribute Routing.

Because we explicitly overrode the default routing for the `BooksController`, the default Tag Helpers (`asp-controller="Books"`) in the layout file will no longer find the correct path. We need to update the link to point to our custom route.

Open `/Views/Shared/_Layout.cshtml`. Look for the "Books Directory" link you added in Lab 2 and change it to use a direct href:

HTML / Razor – /Views/Shared/_Layout.cshtml

```
<li class="nav-item">
    <!-- Updated to use the explicit custom route -->
    <a class="nav-link text-dark" href="/Library/Books">Books Directory</a>
</li>
```

Test the Application

Verifying custom routes and handlers.
Open your VS Code terminal and run:

Terminal / Bash

```
dotnet run
```

Click the URL to open your browser, and perform the following tests to verify your new routing logic:

- Click **Books Directory** in the navigation. Check the URL bar; it should now read `/Library/Books`.
- Add a few books to the directory (e.g., authors like "Tolkien" and "Herbert").
- Use the new **Search by author** box. Look at the URL bar; it should format the request as a query string: `/Library/Books/Search?author=Tolkien`.
- **Test the Route Constraint:** Manually type `/Library/Books/abc/Edit` into your browser's address bar. You will receive a blank page or a 404 error because `abc` is not an integer.
- **Test the JSON API:** Manually type `/Library/Books/api/all` into your address bar. Instead of a webpage, you will see your books rendered as raw JSON data.